

A Student Reflection on the Foundations of Natural Language Processing: From Tokenization to Semantic Embeddings

Carl Angelo Patano
Information Technology
Jose Rizal University
Mandaluyong City, Philippines
carlangelo.patano@my.jru.edu

Abstract—This paper is my reflection on the module Introduction to Natural Language Processing Concepts and the short hands-on text analytics activity that goes with it. Before taking the module I only had a general idea of what NLP was, based mostly on apps I use every day such as translators and chatbots. I could name a few terms but I could not really explain how a computer turns plain text into something it can work with. The module helped me see text analysis as a set of clear steps that build on each other, starting with tokenization, moving to word counting and weighting, and ending with vector models that capture meaning. In this report I go over what I learned in each step, what I noticed while doing the exercise, and how I think these basics will help me in my later subjects, laboratory work, and capstone project as an Information Technology student.

Keywords—natural language processing, tokenization, TF-IDF, word embeddings, text analytics, reflection

I. INTRODUCTION

Natural language processing, usually shortened to NLP, is the area of artificial intelligence that lets computers work with human language. Before I started this module I had a rough idea of what it was, mostly from using tools like online translators and chat assistants, but I would not have been able to explain what happens inside them. I had heard the word tokens and I knew embeddings were important, yet if a classmate asked me to define either one I would have had a hard time.

The module changed that for me. It goes through text analysis one stage at a time. It begins by breaking text into small pieces, then shows how those pieces are counted and weighted, and finally explains how words can be turned into numbers, called vectors, that hold their meaning. The point that stayed with me is simple: no matter how basic or advanced a method is, it is really just trying to pull meaning out of ordinary text. This report follows that same order. I start with what I learned, then talk about what I saw in the exercise, and end with how these ideas connect to the rest of my course.

II. EXPERIENTIAL AND PRACTICAL INSIGHTS

For the hands-on portion I worked through the Explore text analytics exercise using the Language Playground. One of the tasks I tried was language detection. I pasted in a short Japanese paragraph, and the tool correctly identified the

language as Japanese with a prediction confidence of 83 percent, as shown in Fig. 1. I also looked at how it handled key phrase extraction, entity recognition, and sentiment on other samples. Seeing the input and the output next to each other made the earlier lessons feel much more concrete than just reading about them.

A few things stood out to me. The first is that cleaning up the text is not a small detail. When the words were set to lowercase and the punctuation was taken out, the list of key terms that came back was not the same, which showed me that these early steps really do affect the result. The second is that plain word counting stops being useful once you compare more than one document, because the most common words rise to the top and cover up what actually makes each document different. That is the exact weakness that TF-IDF is meant to fix. The third is that measuring how close two pieces of text are using vectors gave me results that matched my own sense of which words go together, which is something simple keyword matching cannot do.

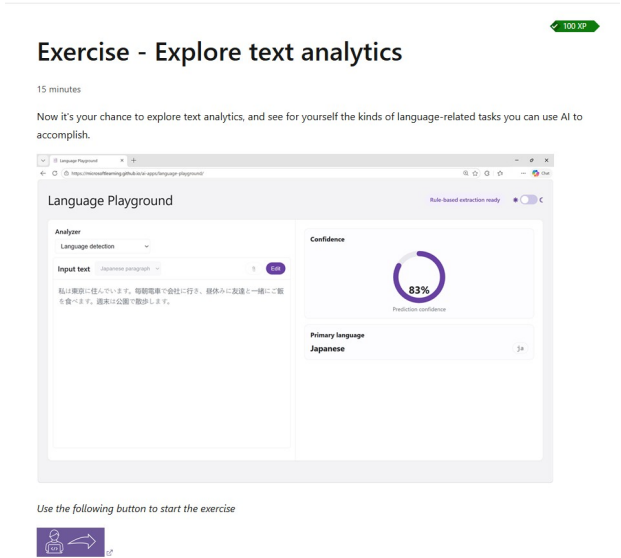


Fig. 1. Language detection in the Language Playground during the Explore text analytics exercise, showing an input paragraph correctly identified as Japanese with 83% prediction confidence.

III. CORE LEARNING OUTCOMES

A. Tokenization and Its Trade-offs

Tokenization is the first step, and it means breaking a body of text into tokens, where each token is usually a word that is given a number so it can be counted. What I learned is that even this early step involves choices. Turning everything to lowercase and removing punctuation helps when you only care about how often words appear, but it can also throw away meaning. The module gives the example of the sentence about Mr Banks, where the name of the person and the word for the banks he worked in would be treated as the same thing. Other options include removing common stop words like the and a, grouping words into phrases such as natural language processing, and cutting words down to a base form through stemming or lemmatization so that running becomes run. There is no single correct setup. The right choice depends on the task you are trying to solve.

B. TF-IDF Compared with Embeddings

The next idea is a way of scoring words called Term Frequency-Inverse Document Frequency, or TF-IDF. It takes how often a word shows up in one document and balances it against how common that word is across all the documents, using the formula $tf \times \log(N/df)$, where N is the total number of documents and df is how many of them contain the word. If a word appears in every document its score becomes zero, so it adds nothing useful. TF-IDF is helpful, but it only looks at the words themselves, so it has no way of knowing that car and automobile mean almost the same thing. Embeddings, from methods like Word2Vec and GloVe, solve this by turning each word into a vector whose direction stands for its meaning. Words with similar meanings point in similar directions, which allows comparisons like cosine similarity and even simple arithmetic, such as dog plus young landing close to puppy. Seeing the difference between counting words and representing their meaning was the clearest lesson in the whole module for me.

C. Context Vectors

The last idea builds on embeddings. In the basic version every word has one fixed vector no matter where it appears. A newer approach called attention creates vectors that change based on the words around them, so the same word can be represented differently depending on the sentence. These context aware vectors are what modern models in the GPT family are built on. Learning this helped me understand how today's language models moved past fixed word lists toward meaning that depends on context.

IV. COURSE APPLICATION AND IMPACT

As an Information Technology student, I can already see where these basics will matter. The lessons on tokenization

and text cleaning will be useful in the data preparation part of future laboratory activities, since the quality of the input shapes everything that comes after it. Knowing the difference between TF-IDF and embeddings also gives me a real reason to choose one method over another in my capstone project, instead of just picking whatever library is easiest to install. More generally, looking at NLP as a series of clear steps has changed how I think about mistakes. When a result looks wrong, I now try to figure out which stage, whether it is the tokenizing, the way the text is represented, or the model itself, is most likely the cause.

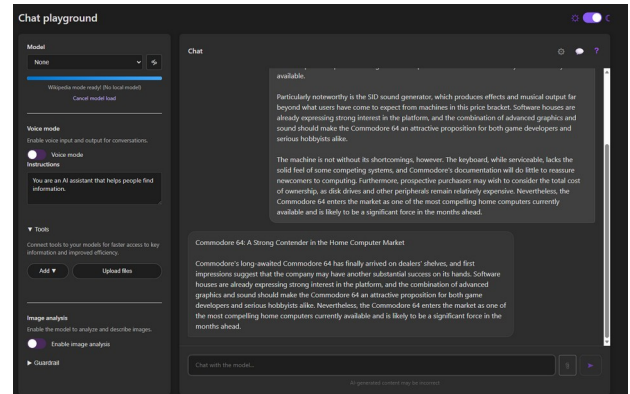


Fig. 2. Generating a short news-style article in the Chat playground. Applying a language model to produce and summarize text shows the kind of NLP task these concepts lead toward in later labs and the capstone project.

V. CONCLUSION AND FUTURE WORK

Finishing this module turned NLP from something that felt like a mystery into a process I can follow. My main takeaways are that the early cleaning steps really do change the outcome, that TF-IDF and embeddings represent two different ways of looking at text, and that context vectors connect the older counting methods to the generative models used today. Going forward, I want to study the transformer design and attention in more detail, try out fine-tuning and retrieval based approaches, and learn more about how to test language models and use them responsibly. These are the areas I plan to explore as I move deeper into my program.

ACKNOWLEDGMENT

I would like to acknowledge the Microsoft Learn module Introduction to Natural Language Processing Concepts, which served as the main source for the technical content discussed in this reflection.

REFERENCES

- [1] Microsoft Learn, "Introduction to natural language processing concepts," Microsoft, 2025. [Online]. Available: <https://learn.microsoft.com/en-us/training/modules/introduction-language/>